Debugging and **Testing**

1️⃣ What is Debugging?

Debugging is the process of identifying, analyzing, and fixing errors (bugs) in your JavaScript code.

Types of Errors in JavaScript

1. **Syntax Errors** – Incorrect code structure
2. **Runtime Errors** – Errors during execution
3. **Logical Errors** – Code runs but gives wrong output

🔹 Example 1: Syntax Error

```
let a = 10

let b = 20

console.log(a + b   // Missing closing bracket
```

👉 Browser Console Output:

```
Uncaught SyntaxError: missing ) after argument list
```

✓ Fix:

```
console.log(a + b);
```

🔹 Example 2: Runtime Error

```
let user = null;

console.log(user.name);
```

👉 Error:

```
Cannot read properties of null
```

✓ Fix:

```
if (user) {
```

```
  console.log(user.name);
}
```

🔹 **Example 3: Logical Error**

```
function multiply(a, b) {
  return a + b;   // Wrong logic
}

console.log(multiply(5, 3));  // Expected 15 but gives 8
```

✓ Fix:

```
return a * b;
```

2️⃣ Debugging Tools & Techniques

🔹 **1. console.log() Debugging**

Most basic and commonly used.

```
function add(a, b) {
  console.log("a:", a);
  console.log("b:", b);
  return a + b;
}

add(5, 10);
```

Other Console Methods

```
console.error("This is error");
console.warn("Warning message");
```

```
console.table([{name:"A"}, {name:"B"}]);

console.time("timer");

// some code

console.timeEnd("timer");
```

🔹 2. Using Browser Developer Tools (Chrome)

How to Open:

- Press **F12**
- OR Right Click → Inspect → Console

Main Tabs:

- Console
- Sources (Debugger)
- Network
- Application

🔹 3. Breakpoints (Professional Debugging)

Instead of console.log everywhere, use breakpoints.

Steps:

1. Open DevTools → Sources tab
2. Click on JS file
3. Click line number → Adds breakpoint
4. Refresh page

Execution pauses at that line.

You can:

- Inspect variables
- Step Over

- Step Into
- Step Out
- Resume execution

🔹 4. Using debugger Keyword

```
function test() {
  let x = 10;
  let y = 20;
  debugger;   // Execution pauses here
  let z = x + y;
  console.log(z);
}

test();
```

If DevTools is open, execution stops at debugger.

3️⃣ Debugging DOM Code (Practical Demo)

```
<input type="text" id="name">
<button onclick="show()">Show</button>

<script>
function show() {
  let value = document.getElementById("nam").value; // Wrong ID
  console.log(value);
}
</script>
```

Error:

Cannot read properties of null

✓ Fix:

```
document.getElementById("name").value;
```

4️⃣ What is Testing?

Testing ensures your code works correctly and continues to work after changes.

There are two main types:

1. Manual Testing
2. Automated Testing

5️⃣ Manual Testing

Run the function and check output manually.

```
function add(a, b) {
  return a + b;
}

console.log(add(2,3));  // Should be 5
```

Problem:

- Not scalable
- Not reliable for big apps

6️⃣ Automated Testing in JavaScript

Professional apps use testing frameworks like:

- Jest
- Mocha

- Jasmine

🔹 **Example: Testing with Jest**

Install:

npm install jest

add.js

```
function add(a, b) {
  return a + b;
}

module.exports = add;
```

add.test.js

```
const add = require('./add');

test('adds 2 + 3 to equal 5', () => {
  expect(add(2,3)).toBe(5);
});
```

Run:

npm test

Output:

✓ adds 2 + 3 to equal 5

7️⃣ Unit Testing Concept

Unit testing means:
Testing small pieces of code independently.

Example:

```
function isEven(num) {
  return num % 2 === 0;
}
```

Test cases:

- isEven(2) → true
- isEven(5) → false
- isEven(0) → true

8️⃣ Assertion Example (Without Framework)

```
function assertEqual(actual, expected) {
  if (actual !== expected) {
    console.error("Test Failed");
  } else {
    console.log("Test Passed");
  }
}

assertEqual(2+2, 4);
assertEqual(2+2, 5);
```

9️⃣ Debugging vs Testing (Important Difference)

Debugging	Testing
Fixing errors	Finding errors
Manual process	Automated possible
Happens after bug found	Happens before bug occurs

Debugging	Testing
Uses DevTools	Uses test frameworks

🔟 Best Practices (Industry Level)

✓ Use strict mode

```
"use strict";
```

✓ Always handle errors

```
try {
  riskyCode();
} catch (err) {
  console.error(err.message);
}
```

✓ Write unit tests for:

- Utility functions
- API logic
- Business logic

✓ Use ESLint (static code analysis)

✓ Use TypeScript for safer code